BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI

Semester: 5

Academic Year: 2026-27

Practical No: 3

Division: C Batch: 2

Roll no: 3090341 PRN No: 12410595

Name of Student: Pranav Khatod

Subject Name : Deep Learning

Problem Statement:

Implement forward propagation and backpropagation using TensorFlow/Keras. Analyze the effect of different learning rates and the number of epochs on model performance.

Algorithm :

Libraries Used:
1. TensorFlow (tensorflow)
2. Matplotlib (matplotlib.pyplot)
3. Pandas

Data Preparation

- We use standard MNIST dataset.
- The input image data is flattened into a 1D array of length **784** and normalized to a scale of 0 to 1 by dividing the pixel values by **255.0**.

Model Architecture
The function create_model(lr) is defined to construct a Sequential neural network with the following architecture:

- **Input/Hidden Layer 1**: A Dense layer with **128** neurons utilizing ReLU activation.

- **Hidden Layer 2**: A Dense layer with **64** neurons utilizing ReLU activation.
- **Output Layer**: A Dense layer with **10** neurons utilizing Softmax activation to handle multi-class classification.

The model is compiled using the Adam optimizer with a customizable learning rate (lr), computing the sparse_categorical_crossentropy loss, and tracking accuracy metrics.

Experiments and Results

The notebook evaluates the model's performance by systematically testing specific hyperparameters:

1. Learning Rate Optimization The model is trained for **5** epochs using a batch size of **128** to test the impact of different learning rates.

- A learning rate of **0.1** achieved an accuracy of **0.6923**.
- A learning rate of **0.01** achieved an accuracy of **0.9698**.
- A learning rate of **0.001** yielded the best initial result with an accuracy of **0.9751**.

2. Epoch Optimization Using the optimal learning rate of **0.001**, the model is then tested over varying training epochs, utilizing a validation split of **0.2** (or 20%).

- Training for **5** epochs resulted in an accuracy of **0.9717**.
- Training for **10** epochs resulted in an accuracy of **0.9761**.
- Training for **20** epochs resulted in an accuracy of **0.9781**.

Finally, the code plots a full training cycle over **20** epochs to visualize the history of both accuracy and loss across the training and validation sets, ultimately achieving a final evaluation accuracy on the test data of approximately **0.9731**

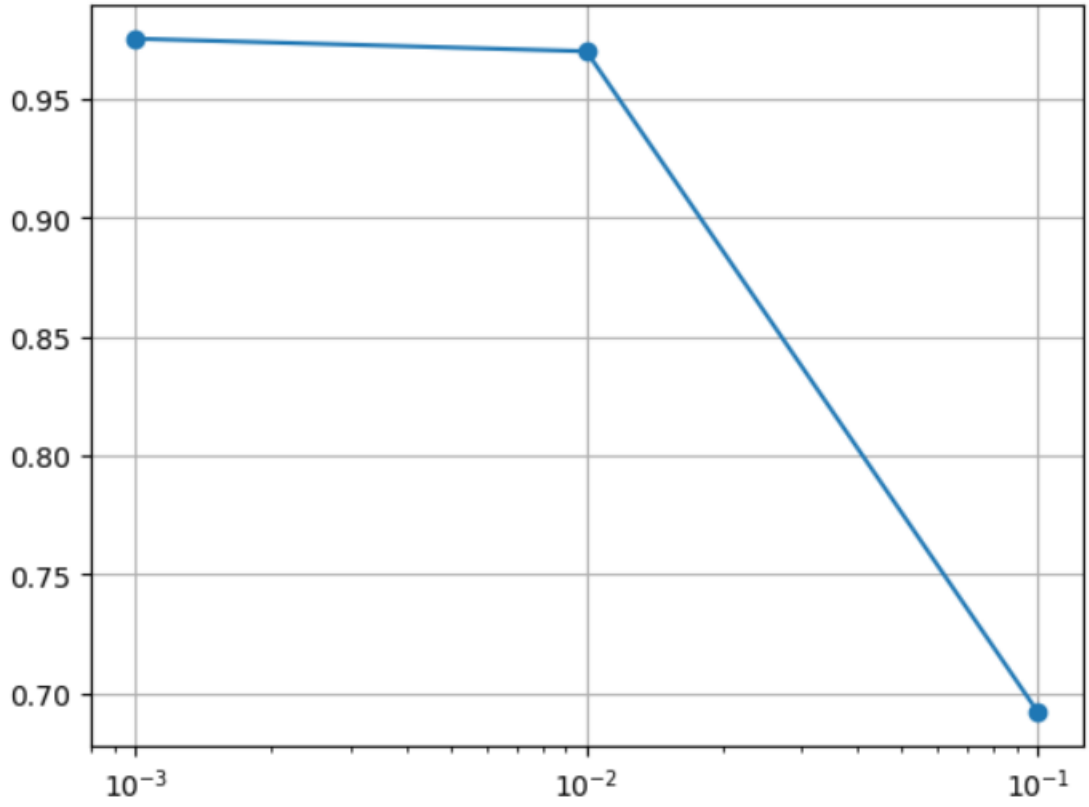
Code :

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
import matplotlib.pyplot as plt
import pandas as pd
(x_train,y_train),(x_test,y_test)=tf.keras.datasets.mnist.load_data()
x_train=x_train.reshape(-1,784)/255.0
x_test=x_test.reshape(-1,784)/255.0
def create_model(lr):
    m=Sequential([Dense(128,activation='relu',input_shape=(784,)),
                  Dense(64,activation='relu'),
                  Dense(10,activation='softmax')])
    m.compile(optimizer=Adam(learning_rate=lr),
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
    return m

learning_rates=[0.1,0.01,0.001]
results=[]
for lr in learning_rates:
    model=create_model(lr)
```

```
    model.fit(x_train,y_train,epochs=5,batch_size=128,verbose=0)
    loss,acc=model.evaluate(x_test,y_test,verbose=0)
    results.append([lr,acc])
df=pd.DataFrame(results,columns=['Learning Rate','Accuracy'])
print(df)
plt.plot(df['Learning Rate'],df['Accuracy'],marker='o')
plt.xscale('log'); plt.grid(); plt.show()
```

```
   Learning Rate  Accuracy
0          0.100    0.6923
1          0.010    0.9698
2          0.001    0.9751
```

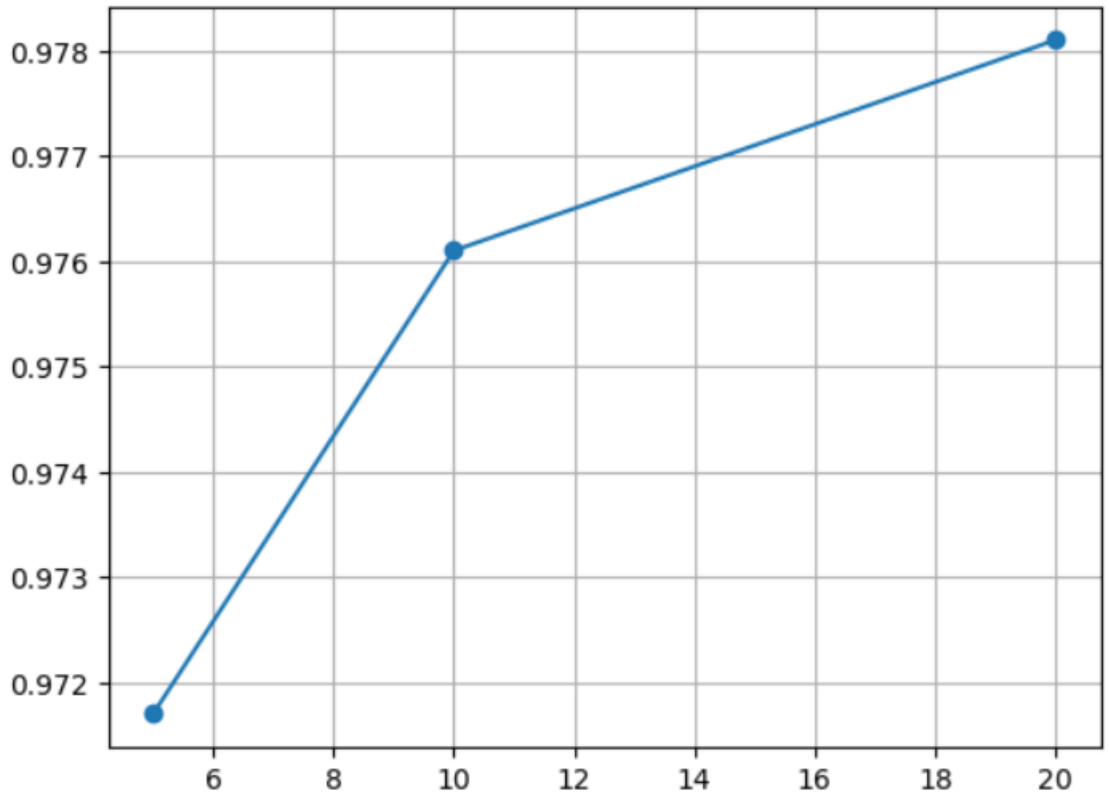



```
epochs_list=[5,10,20]
res=[]
for ep in epochs_list:
    model=create_model(0.001)
    h=model.fit(x_train,y_train,validation_split=0.2,epochs=ep,batch_size=128,verbose=0)
    loss,acc=model.evaluate(x_test,y_test,verbose=0)
    res.append([ep,acc])
edf=pd.DataFrame(res,columns=['Epochs','Accuracy'])
print(edf)
plt.plot(edf['Epochs'],edf['Accuracy'],marker='o')
plt.grid(); plt.show()

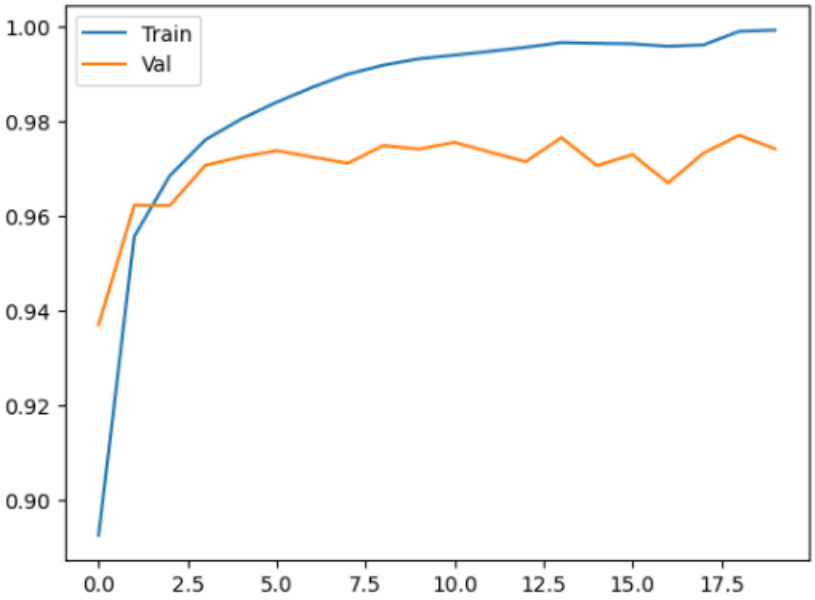
model=create_model(0.001)
history=model.fit(x_train,y_train,validation_split=0.2,epochs=20,batch_size=128)
```

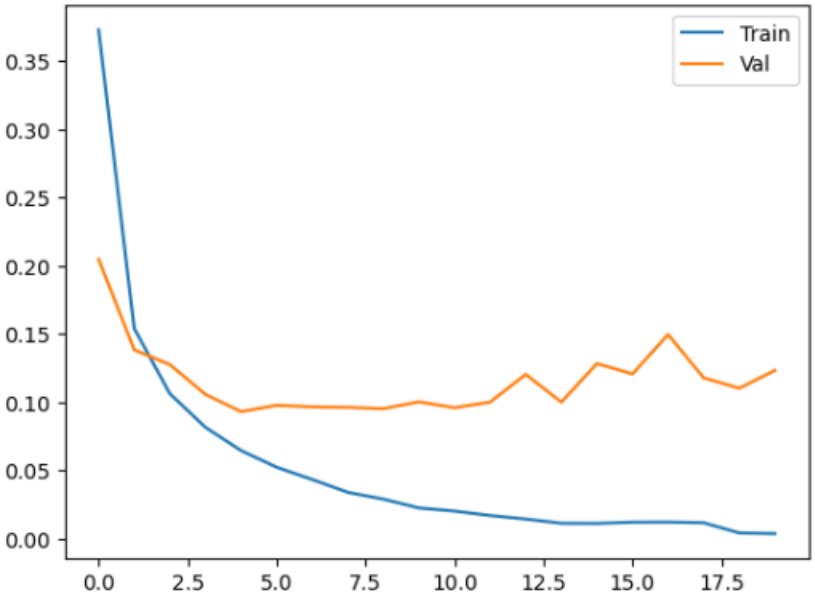
```
plt.plot(history.history['accuracy']); plt.plot(history.history['val_accuracy']); plt.legend(['Train','Val']);
plt.show()
plt.plot(history.history['loss']); plt.plot(history.history['val_loss']); plt.legend(['Train','Val']); plt.show()
print(model.evaluate(x_test,y_test))
```

	Epochs	Accuracy
0	5	0.9717
1	10	0.9761
2	20	0.9781



```
Epoch 1/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 9s 15ms/step - accuracy: 0.8927 - loss: 0.3731 - val_accuracy: 0.9371 - val_loss: 0.2049
Epoch 2/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 13ms/step - accuracy: 0.9557 - loss: 0.1541 - val_accuracy: 0.9623 - val_loss: 0.1385
Epoch 3/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - accuracy: 0.9685 - loss: 0.1063 - val_accuracy: 0.9622 - val_loss: 0.1276
Epoch 4/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 10ms/step - accuracy: 0.9761 - loss: 0.0816 - val_accuracy: 0.9707 - val_loss: 0.1058
Epoch 5/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 10ms/step - accuracy: 0.9805 - loss: 0.0646 - val_accuracy: 0.9725 - val_loss: 0.0931
Epoch 6/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 6s 11ms/step - accuracy: 0.9841 - loss: 0.0524 - val_accuracy: 0.9738 - val_loss: 0.0978
Epoch 7/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - accuracy: 0.9872 - loss: 0.0434 - val_accuracy: 0.9725 - val_loss: 0.0966
Epoch 8/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 10ms/step - accuracy: 0.9900 - loss: 0.0340 - val_accuracy: 0.9712 - val_loss: 0.0963
Epoch 9/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.9919 - loss: 0.0290 - val_accuracy: 0.9749 - val_loss: 0.0953
Epoch 10/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.9932 - loss: 0.0225 - val_accuracy: 0.9742 - val_loss: 0.1003
Epoch 11/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.9940 - loss: 0.0202 - val_accuracy: 0.9756 - val_loss: 0.0960
Epoch 12/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - accuracy: 0.9948 - loss: 0.0170 - val_accuracy: 0.9735 - val_loss: 0.1002
Epoch 13/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 13ms/step - accuracy: 0.9956 - loss: 0.0144 - val_accuracy: 0.9715 - val_loss: 0.1205
Epoch 14/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 13ms/step - accuracy: 0.9966 - loss: 0.0113 - val_accuracy: 0.9766 - val_loss: 0.1001
Epoch 15/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 13ms/step - accuracy: 0.9965 - loss: 0.0112 - val_accuracy: 0.9707 - val_loss: 0.1284
Epoch 16/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 13ms/step - accuracy: 0.9964 - loss: 0.0121 - val_accuracy: 0.9730 - val_loss: 0.1208
Epoch 17/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 6s 15ms/step - accuracy: 0.9959 - loss: 0.0122 - val_accuracy: 0.9670 - val_loss: 0.1497
Epoch 18/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 13ms/step - accuracy: 0.9962 - loss: 0.0117 - val_accuracy: 0.9733 - val_loss: 0.1179
Epoch 19/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 14ms/step - accuracy: 0.9990 - loss: 0.0042 - val_accuracy: 0.9771 - val_loss: 0.1103
Epoch 20/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 13ms/step - accuracy: 0.9992 - loss: 0.0037 - val_accuracy: 0.9743 - val_loss: 0.1234
```






```
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9731 - loss: 0.1198
[0.11983692646026611, 0.9731000065803528]
```